SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	**CSA1209 & CSA1215**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO3**	**Assessment:**	**ASSESSMENT TOOL 1- EXPERIMENTS**
Weightage:	**40%**	**Total Marks:**	**25**
CO3 Statement:	*Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)*		
Student Name:	Deepika shree S	**Reg.No.:**	**192521264**
Department:	Btech IT	**Date:**	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	**Experimental Design (20%)**	**Use of Tools (25%)**	**Analysis of Results (20%)**	**Documentation (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. REGISTER TRANSFER OPERATIONS AND ALU OPERATIONS

AIM

To simulate register transfer operations and basic arithmetic and logical operations using an Arithmetic Logic Unit (ALU).

SOFTWARE REQUIRED

Python 3.x

OBJECTIVE

To understand how data is transferred between registers and how the ALU performs arithmetic and logical operations.

THEORY

Register transfer refers to the movement of data from one register to another. The ALU performs arithmetic and logical operations on data received from registers. In this experiment, addition, subtraction, AND, and OR operations are performed.

PROCEDURE

1. Initialize multiple registers with suitable values.
2. Transfer data between the registers.
3. Select the required ALU operation.
4. Perform addition, subtraction, AND, and OR operations.
5. Store the results in destination registers.
6. Display the register contents and ALU results.

PROGRAM

```
registers = {"R1": 10, "R2": 5, "R3": 0, "R4": 0, "R5": 0,"R6": 0}
print("Initial Registers:") print(registers)
registers["R3"] = registers["R1"] + registers["R2"]
registers["R4"] = registers["R1"] - registers["R2"] registers["R5"]
= registers["R1"] & registers["R2"] registers["R6"] =
registers["R1"] | registers["R2"]
print("\nALU Results:") print("R3 = R1 +
R2 =", registers["R3"]) print("R4 = R1 - R2
=", registers["R4"]) print("R5 = R1 AND R2
=", registers["R5"]) print("R6 = R1 OR R2
=", registers["R6"]) OUTPUT
```

main.c

```c
#include <stdio.h>
int main()
{
    int R1, R2, R3, result;
    printf("Enter value for R1: ");
    scanf("%d", &R1);
    printf("Enter value for R2: ");
    scanf("%d", &R2);
    R3 = R1;
    printf("\nRegister Transfer: R3 <- R1");
    printf("\nR3 = %d\n", R3);
    result = R1 + R2;
    printf("\nAddition (R1 + R2) = %d", result);
    result = R1 - R2;
    printf("\nSubtraction (R1 - R2) = %d", result);
    result = R1 & R2;
    printf("\nAND (R1 & R2) = %d", result);
    result = R1 | R2;
    printf("\nOR (R1 | R2) = %d\n", result);
    return 0;
}
```

```
Initial Registers:
{'R1': 10, 'R2': 5, 'R3': 0, 'R4': 0, 'R5': 0, 'R6': 0}

ALU Results:
R3 = R1 + R2 = 15
R4 = R1 - R2 = 5
R5 = R1 AND R2 = 0
R6 = R1 OR R2 = 15
```

RESULT

The register transfer operations and ALU operations such as addition, subtraction, AND, and OR were successfully simulated using Python.

2. SINGLE-BUS AND MULTI-BUS ORGANIZATION

AIM

To model single-bus and multi-bus computer organization and compare their data transfer efficiency.

SOFTWARE REQUIRED

Python 3.x

OBJECTIVE

To understand how bus organization affects data transfer time and processor performance.

THEORY

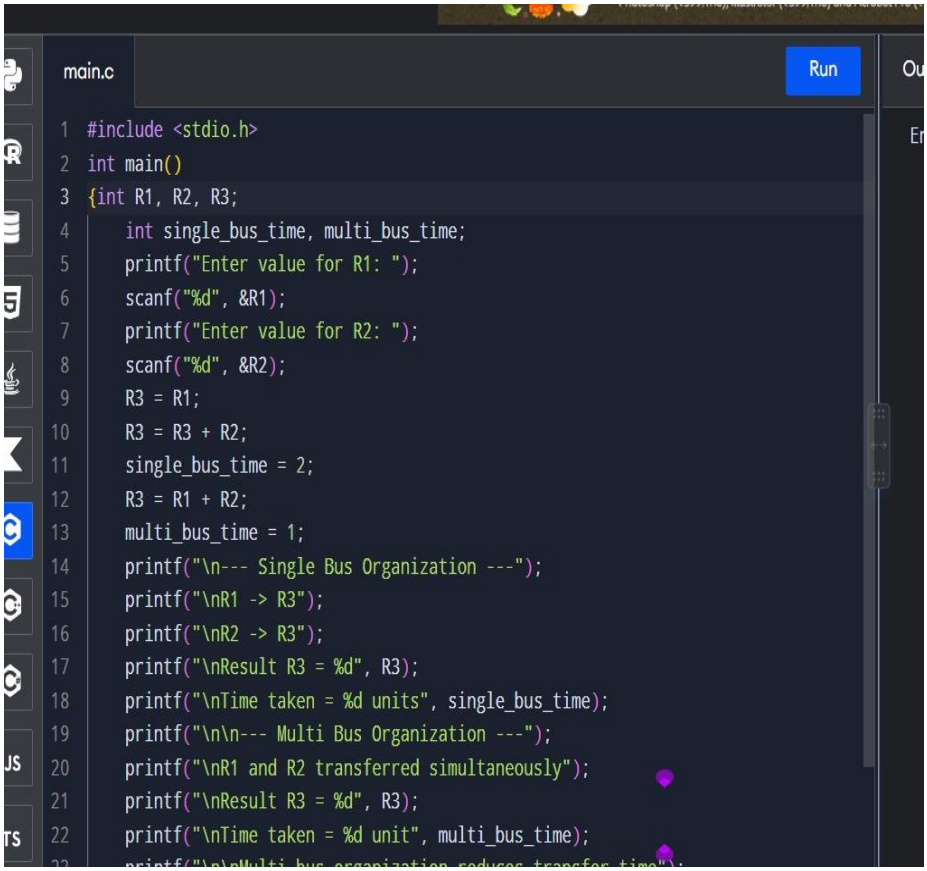
A single-bus organization uses one common bus for transferring data between registers, memory, and I/O devices. Only one major transfer can occur at a time. A multi-bus organization uses multiple buses, allowing several data transfers to occur simultaneously and reducing bottlenecks.

PROCEDURE

1. Initialize registers and memory locations.
2. Perform data transfers using a single-bus system.
3. Count the number of transfer cycles.
4. Perform the same operations using a multi-bus system.
5. Count the required cycles.
6. Compare the execution time and efficiency.

PROGRAM

```
transfers = ["Memory -> R1","R1 -> R2","R2 -> R3", "R3 -> Memory"]
single_bus_cycles = len(transfers) multi_bus_cycles = 2 print("Single-
Bus Organization") for i, t in enumerate(transfers, 1):
   print("Cycle", i, ":", t)
print("\nSingle-Bus Cycles =", single_bus_cycles)
print("Multi-Bus Cycles =", multi_bus_cycles)
speedup = single_bus_cycles / multi_bus_cycles
print("Speedup =", speedup)
```



OUTPUT

```
Single-Bus Organization
Cycle 1 : Memory -> R1
Cycle 2 : R1 -> R2
Cycle 3 : R2 -> R3
Cycle 4 : R3 -> Memory

Single-Bus Cycles = 4
Multi-Bus Cycles = 2
Speedup = 2.0
```

RESULT

The multi-bus organization required fewer cycles than the single-bus organization. Therefore, multiple buses improve parallel data transfer and reduce data-transfer bottlenecks.

3. FIVE-STAGE INSTRUCTION PIPELINE

AIM

To simulate a five-stage instruction pipeline and compare its performance with a nonpipelined processor.

SOFTWARE REQUIRED

Python 3.x

OBJECTIVE

To understand instruction pipelining and evaluate its effect on execution time, speedup, and throughput.

THEORY

A typical five-stage instruction pipeline consists of:

1. IF – Instruction Fetch
2. ID – Instruction Decode
3. EX – Execute
4. MEM – Memory Access
5. WB – Write Back

Pipelining allows multiple instructions to be processed simultaneously at different stages.

PROCEDURE

1. Define a set of instructions.
2. Divide instruction execution into five pipeline stages.
3. Execute the instructions using a pipelined approach.
4. Calculate the number of pipeline cycles.
5. Calculate the non-pipelined execution cycles.
6. Determine speedup and throughput.

PROGRAM

```
instructions = ["I1", "I2", "I3", "I4", "I5"] stages =
["IF", "ID", "EX", "MEM", "WB"] pipeline_cycles
= len(instructions) + len(stages) - 1
non_pipeline_cycles = len(instructions) * len(stages)
speedup = non_pipeline_cycles / pipeline_cycles
throughput = len(instructions) / pipeline_cycles
print("Pipeline Execution") for i in
range(len(instructions)):     print(instructions[i],
"starts at cycle", i + 1) print("\nPipelined Cycles =",
pipeline_cycles) print("Non-Pipelined Cycles =",
non_pipeline_cycles) print("Speedup =",
round(speedup, 2))
print("Throughput =", round(throughput, 3), "instructions/cycle")
```

main.c

Run

```c
#include <stdio.h>
int main()
{
    int n, i;
    int non_pipelined, pipelined;
    float speedup, throughput;
    printf("Enter number of instructions: ");
    scanf("%d", &n);
    non_pipelined = n * 5;
    pipelined = 5 + n - 1;
    speedup = (float)non_pipelined / pipelined;
    throughput = (float)n / pipelined;
    printf("\n5-Stage Pipeline:");
    printf("\n1. Fetch");
    printf("\n2. Decode");
    printf("\n3. Execute");
    printf("\n4. Memory");
    printf("\n5. Write-back");
    printf("\n\nNon-Pipelined Cycles = %d", non_pipelined);
    printf("\nPipelined Cycles     = %d", pipelined);
    printf("\nSpeedup              = %.2f", speedup);
    printf("\nThroughput           = %.2f instructions/cycle\n", throughput);
```

OUTPUT

```
Pipeline Execution
I1 starts at cycle 1
I2 starts at cycle 2
I3 starts at cycle 3
I4 starts at cycle 4
I5 starts at cycle 5

Pipelined Cycles = 9
Non-Pipelined Cycles = 25
Speedup = 2.78
Throughput = 0.556 instructions/cycle
```

RESULT
The five-stage pipeline was successfully simulated. The pipelined system required fewer cycles than the non-pipelined system, demonstrating improved execution performance and throughput.

4. PIPELINE HAZARDS AND HAZARD RESOLUTION

AIM

To simulate data, control, and structural hazards in a pipelined processor and implement techniques to reduce their effects.

SOFTWARE REQUIRED

Python 3.x

OBJECTIVE

To identify different types of pipeline hazards and understand techniques such as stalling, data forwarding, and branch prediction.

THEORY

Pipeline hazards are conditions that prevent the next instruction from executing in its intended clock cycle.

Data Hazard: Occurs when an instruction depends on the result of a previous instruction.

Control Hazard: Occurs due to branch or jump instructions.
Structural Hazard: Occurs when multiple instructions require the same hardware resource simultaneously.

PROCEDURE

1. Create a sequence of dependent instructions.
2. Identify the data hazard between instructions.
3. Apply data forwarding to reduce waiting.
4. Introduce a branch instruction to create a control hazard.
5. Apply branch prediction.
6. Introduce a resource conflict to demonstrate a structural hazard.
7. Compare execution before and after hazard handling.

PROGRAM

```
print("Pipeline Hazard Simulation\n")
print("1. Data Hazard") print("I1: R1
```

= R2 + R3") print("I2: R4 = R1 +
R5") print("Resolution: Data
Forwarding") print("\n2. Control
Hazard") print("I3: BEQ R1, R2")
print("Resolution: Branch Prediction")
print("\n3. Structural Hazard")
print("I4 and I5 require the same memory resource") print("Resolution:
Pipeline Stall")
print("\nHazard handling completed successfully.")

main.c

Run

```c
#include <stdio.h>
int main()
{
    int choice;
    printf("PIPELINE HAZARD SIMULATION\n");
    printf("--------------------------\n");
    printf("1. Data Hazard\n");
    printf("2. Control Hazard\n");
    printf("3. Structural Hazard\n");
    printf("\nEnter your choice: ");
    scanf("%d", &choice);
    switch(choice)
    {
        case 1:
            printf("\nData Hazard Detected");
            printf("\nTechnique Used: Data Forwarding");
            printf("\nStall cycles reduced using forwarding.");
            break;
        case 2:
            printf("\nControl Hazard Detected");
            printf("\nTechnique Used: Branch Prediction");
            printf("\nCorrect prediction reduces pipeline stalls.");
```

OUTPUT

```
Pipeline Hazard Simulation

1. Data Hazard
I1: R1 = R2 + R3
I2: R4 = R1 + R5
Resolution: Data Forwarding

2. Control Hazard
I3: BEQ R1, R2
Resolution: Branch Prediction

3. Structural Hazard
I4 and I5 require the same memory resource
Resolution: Pipeline Stall

Hazard handling completed successfully.
```

RESULT

Data, control, and structural hazards were successfully identified and suitable techniques such as forwarding, branch prediction, and stalling were applied to reduce their impact on pipeline performance.

5. SUPERSCALAR, OUT-OF-ORDER, SPECULATIVE EXECUTION AND SMT

AIM

To simulate advanced processor execution techniques including superscalar execution, outof-order execution, speculative execution, and simultaneous multithreading (SMT).

SOFTWARE REQUIRED

Python 3.x

OBJECTIVE

To understand how modern processors improve instruction throughput by executing multiple instructions and utilizing processor resources efficiently.

THEORY

A superscalar processor can execute multiple instructions in a single clock cycle.

Out-of-order execution allows instructions to execute when their required resources are available rather than strictly following program order.

Speculative execution allows the processor to execute instructions before knowing whether they are actually required.

Simultaneous Multithreading (SMT) allows multiple instruction streams or threads to share processor resources and execute concurrently.

PROCEDURE

1. Create multiple instructions belonging to different threads.
2. Simulate execution of multiple instructions per cycle.
3. Allow independent instructions to execute out of order.
4. Simulate speculative execution for a branch.
5. Execute instructions from multiple threads using SMT.
6. Compare the instruction throughput.

PROGRAM

```
instructions = [
("T1", "I1"),
   ("T1", "I2"),
   ("T2", "I3"),
   ("T2", "I4")
]

print("Superscalar Execution")
print("Cycle 1:", instructions[0], instructions[1])
print("Cycle 2:", instructions[2], instructions[3])

print("\nOut-of-Order Execution")
print("Independent instructions execute when resources are available.")

print("\nSpeculative Execution")
print("Branch instructions are executed based on prediction.")

print("\nSimultaneous Multithreading (SMT)")
print("Thread T1 and Thread T2 execute concurrently.")

total_instructions = len(instructions) cycles
= 2
throughput = total_instructions / cycles

print("\nTotal Instructions =", total_instructions) print("Execution
Cycles =", cycles)
print("Throughput =", throughput, "instructions/cycle")
```

main.c

Run

```c
#include <stdio.h>
int main()
{
    int instructions;
    int superscalar_cycles;
    float throughput;
    printf("Enter number of instructions: ");
    scanf("%d", &instructions);
    superscalar_cycles = (instructions + 1) / 2;
    throughput = (float)instructions / superscalar_cycles;
    printf("\n--- Superscalar Processor ---");
    printf("\nInstructions = %d", instructions);
    printf("\nInstructions issued per cycle = 2");
    printf("\n\nOut-of-Order Execution:");
    printf("\nIndependent instructions are executed first.");
    printf("\n\nSpeculative Execution:");
    printf("\nProcessor predicts future instructions.");
    printf("\n\nSimultaneous Multithreading (SMT):");
    printf("\nMultiple instruction streams use CPU resources.");
    printf("\n\nTotal Cycles = %d", superscalar_cycles);
    printf("\nInstruction Throughput = %.2f instructions/cycle", throughput);
    printf("\n\nResource utilization is improved using");
```

OUTPUT

```
Superscalar Execution
Cycle 1: ('T1', 'I1') ('T1', 'I2')
Cycle 2: ('T2', 'I3') ('T2', 'I4')

Out-of-Order Execution
Independent instructions execute when resources are available.

Speculative Execution
Branch instructions are executed based on prediction.

Simultaneous Multithreading (SMT)
Thread T1 and Thread T2 execute concurrently.

Total Instructions = 4
Execution Cycles = 2
Throughput = 2.0 instructions/cycle
```

RESULT

Superscalar, out-of-order, speculative execution, and SMT concepts were successfully simulated. The results demonstrate that parallel execution techniques can increase instruction throughput and improve processor resource utilization.